

KernelBand: Steering LLM-based Kernel Optimization via Hardware-Aware Multi-Armed Bandits

ICML 2026

Dezhi Ran*^{1 2} Shuxiao Xie*^{2 3} Mingfang Ji^{2 4} Anmin Liu¹ Mengzhou Wu^{1 2} Yuan Cao^{1 2}

Yuzhe Guo^{1 2} Hao Yu⁵ Linyi Li⁶ Yitao Hu⁴ Wei Yanq⁷ Tao Xie^{1 2 8 9}



汇报人：辛金韩

2026年7月13日

Author Team



冉德智

个人主页: <https://dezhi-ran.com/>

北大本科，北大直博，目前博士四年级

研究方向:

- LLM for Software Engineering
- Foundation Model Engineering

近期成果:

- [ICLR' 2026]AppForge: From Assistant to Independent Developer — Are GPTs Ready for Software Development?
- [SCIS' 2025]An Infrastructure Software Perspective toward Computation Offloading between Executable Specifications and Foundation Models.

导师:



谢涛

<https://taoxiease.github.io/>

- 软件工程
- 操作系统
- 可信人工智能
- RISC-V

谢书晓

个人主页: 无

华东师范大学本科生，北京通明湖信息技术应用创新中心实习生

研究方向: 无明确研究方向

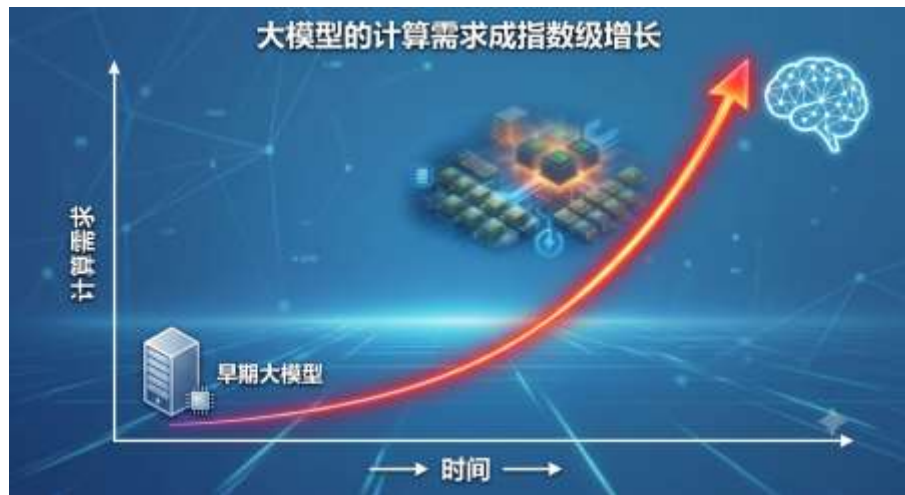
- 欧洲科学院外籍院士
- Fellow of ACM, IEEE, AAAS, CCF
- 北京大学讲席教授
- 北京大学计算机学院软件科学与工程系主任

Contents

- **Background**
- Design
- Evaluation
- Thinking

Background

背景：LLM的计算需求成指数级增长



计算需求高



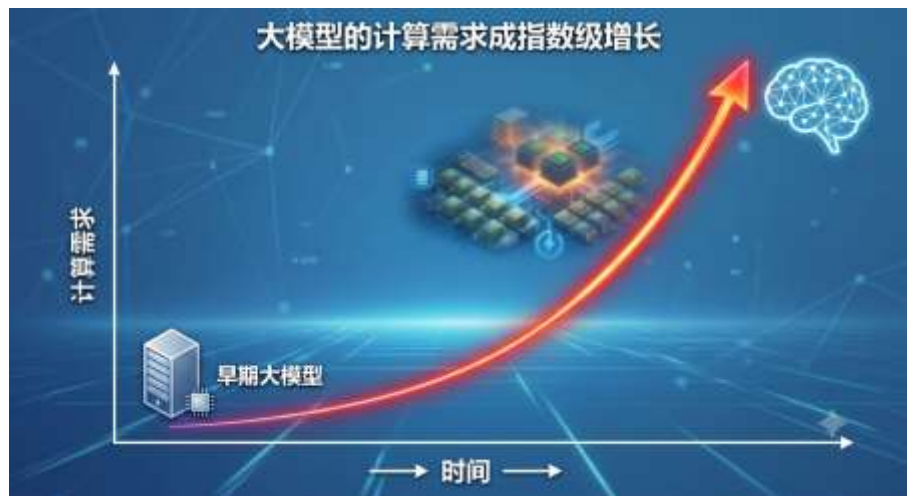
构建高效的服务基础设施成为当务之急

服务基础设施包括：

- GPU/NPU计算资源
- 推理框架：Triton、Pytorch
- 显存管理：KV Cache....
- 底层高性能kernel

Background

背景：LLM的计算需求成指数级增长



计算需求高



构建高效的服务基础设施成为当务之急

其中核心是kernel optimization, 内核优化

Background

Kernel是什么？

维度	算子 (Operator)	Kernel (计算核心)
定位	AI模型中的逻辑计算单元 (功能层面)	算子在特定硬件上的物理实现 (执行层面)
抽象层级	高层 (如卷积、矩阵乘法)	底层 (直接操作 (硬件指令/计算单元))
职责	定义 “做什么” (计算逻辑、输入输出规则)	解决 “怎么做” (硬件指令调度、内存访问优化)
		CUDA Kernel (GPU), GEMV (CPU), Gemv (NPU)

Kernel 是运行在GPU/NPU上的一个计算程序，它描述了 “如何完成一次计算” 。

Background

Kernel优化是什么？

kernel 优化就是在数学结果不变的前提下，把某个底层算子在硬件上跑得更快、更省资源

减少访存开销

提高并行度

利用专用计算
单元

算子融合

调整 tiling /
block size /
pipeline

一个算子对应无数多的kernel，所以说kernel优化是一个搜索问题

Background

传统 kernel 优化难度高

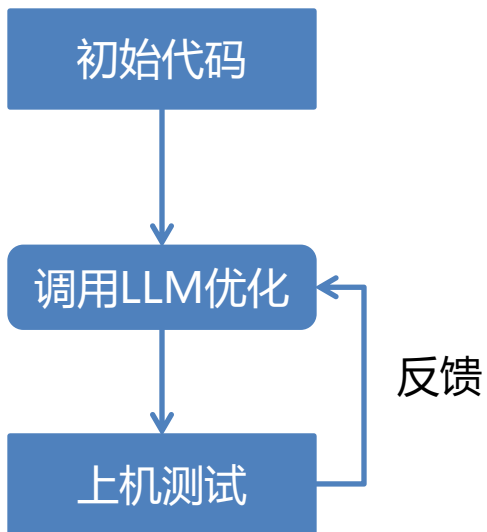
- 专家手工优化成本高：
 - 1.需要经验充足，足够理解硬件细节
 - 2.开发周期长，时间成本高
- DSL+编译器自动调优也难找到全局最优：
 - 1.随硬件复杂度提高，配置空间出现组合爆炸
 - 2.自动调擅长调“参数”，不擅长改“代码结构”



Background

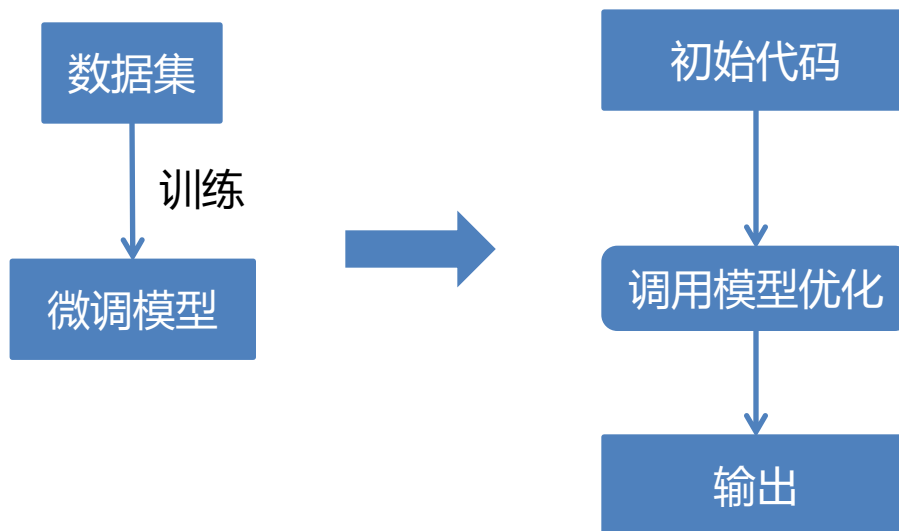
目前研究社区转向使用LLM生成高性能kernel

基于Agent的方法



迭代反馈循环不断改进

基于训练的方法



LLM 擅长生成在统计上高概率且功能正确的代码，却很难在探索与利用之间取得平衡，经常会收敛到次优的局部最小值

Background

因此kernel优化面临两个问题

搜索空间巨大

针对此问题本文提出：

硬件感知剪枝

轨迹驱动聚类

探索-利用平衡问题

针对此问题本文提出：

将优化问题建模为
上下文老虎机

Background

上下文老虎机



多臂老虎机问题里，只有一个老虎机，我们的agent可选行动的范围就是从赌博机的多个臂中选一个拉动。通过这样做，获得一个+1或者-1的回报。当agent学会了总是选择可以返回正回报的臂去拉动时，这个问题就被认为是解决了。

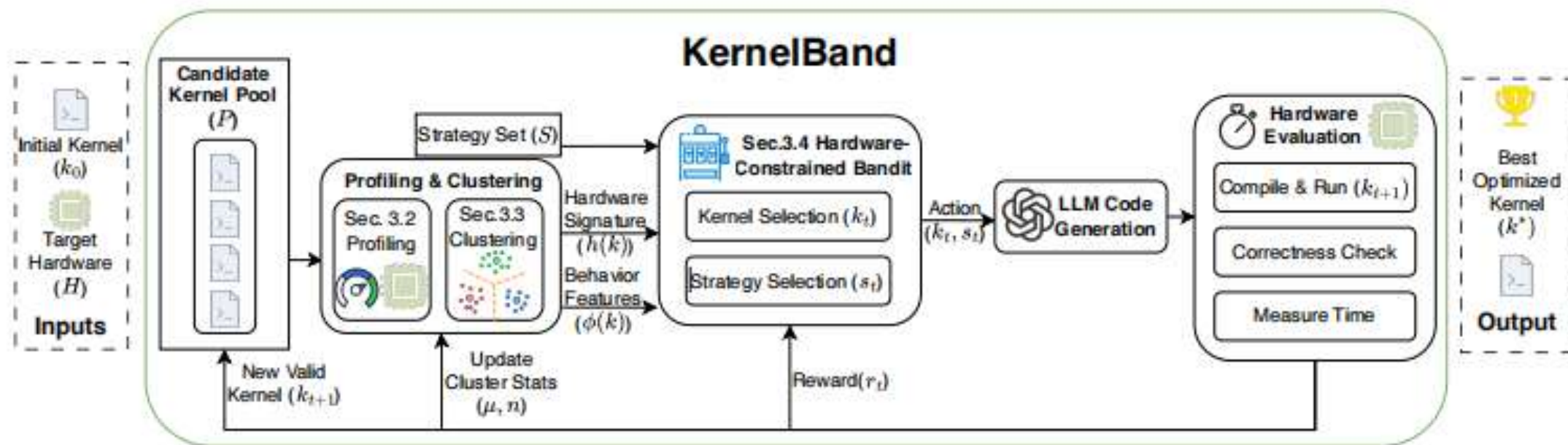


上下文多臂赌博机则是在此基础上加入了"上下文"这一关键元素，也可称为状态。在这个场景中,每次选择前玩家都能获得一些额外的信息(即上下文),这些信息可能会影响不同机器的回报。例如,在推荐系统中,用户的兴趣爱好、浏览历史等就可以作为上下文信息,帮助系统为不同用户推荐最合适的内容。。

Contents

- Background
- **Design**
- Evaluation
- Thinking

Design-Overview



profiling进行策略剪枝+聚类 → 上下文老虎机选择kernel与策略 → LLM生成新kernel+更新候选池、聚类状态及奖励

Design

问题定义:

目标硬件平台 $\mathcal{H}$

+

初始kernel

得到最优性能 k^*

公式: $k^* = \arg \min_{k \in \mathcal{K}} L(k, \mathcal{H})$

$L(k, \mathcal{H})$ 指的是在目标硬件平台该kernel的延迟

为什么是期望奖励后面会讲, 但是这么构建上下文老虎机会有一个问题:

优化过程看作是在一个有向图 $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ 上进行遍历: 节点 $\mathcal{V}$ 表示有效的kernel; 边 $(k \rightarrow k') \in \mathcal{E}$ 表示通过代码LLM将

• 正常老虎机臂固定, 经过反复尝试, 后面就会减少差arm选择, 从而多选好arm, 其累计遗憾为:
某种优化策略 $s \in \mathcal{S}$ 应用于当前kernel, 从而得到新的kernel

$R(T) = o(T)$, 但是是次线性增长, 随迭代次数, 遗憾增加越小, 所以 $R(T)/T$ 平均遗憾会趋于零。

上下文老虎机建模: 由上下文、动作、转移和奖励组成

• 本文里由于迭代次数增加, 候选kernel增加, 动作空间也变大, 这时若按照常规动作去学习, 那么臂

不固定, 累计遗憾可能会呈线性增长, 这样在数学意义上就可以证明无法到达最优值。

上下文空间: 每个候选kernel的特征向量, 内容为静态程序特征+动态执行轨迹。

动作空间: 随迭代次数增加, 内容为某种策略应用于某个候选kernel的集合。

执行动作后, LLM生成的新kernel。

奖励: $r_t \in [0, 1]$, 计算公式为

$$r_t = \text{Clip} \left(\frac{L(k_t) - L(k'_t)}{L(k_t)}, 0, 1 \right)$$

性能退化, 或者编译失败, 则奖励被设为0。智能体的目标是最大化累计期望奖励:

$$\sum_{t=1}^T \mathbb{E}[r_t]$$

Design

kernel 优化问题具有两个关键的结构性质

假设1.优化策略s 应用于kernel k所带来的期望改进，会受到硬件极限的约束。

存在边界函数： $B : \mathcal{V} \times \mathcal{S} \rightarrow [0, 1]$

使得： $\mu(k, s) \triangleq \mathbb{E}[r_t \mid k, s] \leq B(k, s)$

某个优化策略最多能带来多少提升，不是无限的，而是受到硬件瓶颈和理论上限约束的。

假设2.期望奖励函数 $\mu(k, s)$ 关于执行行为是 Lipschitz 连续的

$$|\mu(k_1, s) - \mu(k_2, s)| \leq L \cdot \|\phi(k_1) - \phi(k_2)\|_2$$

$\phi(k)$ 为捕捉的kernel的运行时特征

具有相似瓶颈的 kernel，往往会对相同优化策略产生相似的响应。

Design-Clustering

根据假设2，我们收集了每个候选kernel的行为特征向量 $\phi(k)$

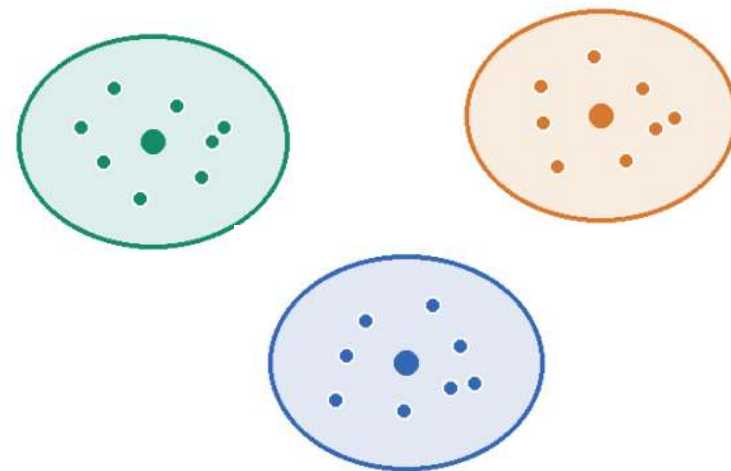
$$\phi(k) = [\tilde{\mathcal{J}}(k), n_{\text{reg}}, n_{\text{smem}}, d_{\text{block}}, \eta_{\text{occ}}]$$

特征	来源
执行时间	运行测试
register	CUDA API
shared memory	CUDA API
block size	launch配置
occupancy	occupancy API

- 执行时间：kernel在目标硬件的执行时间
- register：每线程寄存器数量
- shared memory：每block共享内存大小
- block size：block维度，含多少线程
- occupancy：理论最大活跃warps/最大warps

周期性聚类

每隔 τ 次迭代，系统使用K-Means对所有候选kernel的行为特征 $\phi(k)$ 进行聚类，将当前候选kernel集合划分为K个簇。 $\mathcal{C}_t = \{C_1, \dots, C_K\}$



- 1.需要跟踪kernel行为随优化过程发生的变化；
- 2.它也需要保持足够稳定，以满足收敛所需要的统计稳定性。

Design-Pruning

根据假设1，策略优化上限受硬件约束



通过profiling收集kernel的硬件签名 $h(k)$

$h(k)$ 测量以下硬件资源的峰值吞吐率

DRAM

L2
cache

SM

用于识别当前 kernel 的主导瓶颈，计算、cache和内存带宽瓶颈

代表性 profiling

由于硬件profiling的成本较高，大约需要10秒，因此在重新聚类阶段，只profiling每个活跃簇的中心kernel。

策略剪枝

$$M_{i,s} = \mathbb{I} \left[h(k_c^{(i)})[\text{Target}(s)] < \theta_{\text{sat}} \right]$$

其中 $M_{i,s} \in \{0, 1\}$

公式大意，如果策略在该质心kernel上对应优化的硬件资源接近饱和阈值 θ_{sat} ，则掩码为零，后面策略选择便不考虑该策略。

Design-Action selection

使用UCB算法选择一个“簇-策略”组合(I_t, S_t)

$$(I_t, S_t) = \underset{\substack{i \in [K], s \in \mathcal{S} \\ \text{s.t. } M_{i,s} = 1}}{\operatorname{argmax}} \left(\hat{\mu}_{i,s} + c \sqrt{\frac{\ln t}{N_{i,s}}} \right)$$

$\hat{\mu}_{i,s}$ 表示对某个簇的某个策略组合的平均奖励

$N_{i,s}$ 表示某个簇的某个策略组合的访问次数

利用和探索的权衡

选完后, KernelBand 会从簇中采样一个具体 kernel:

$$V_{hw}(k, S_t) = \theta_{sat} - h(k)[\text{Target}(S_t)]$$

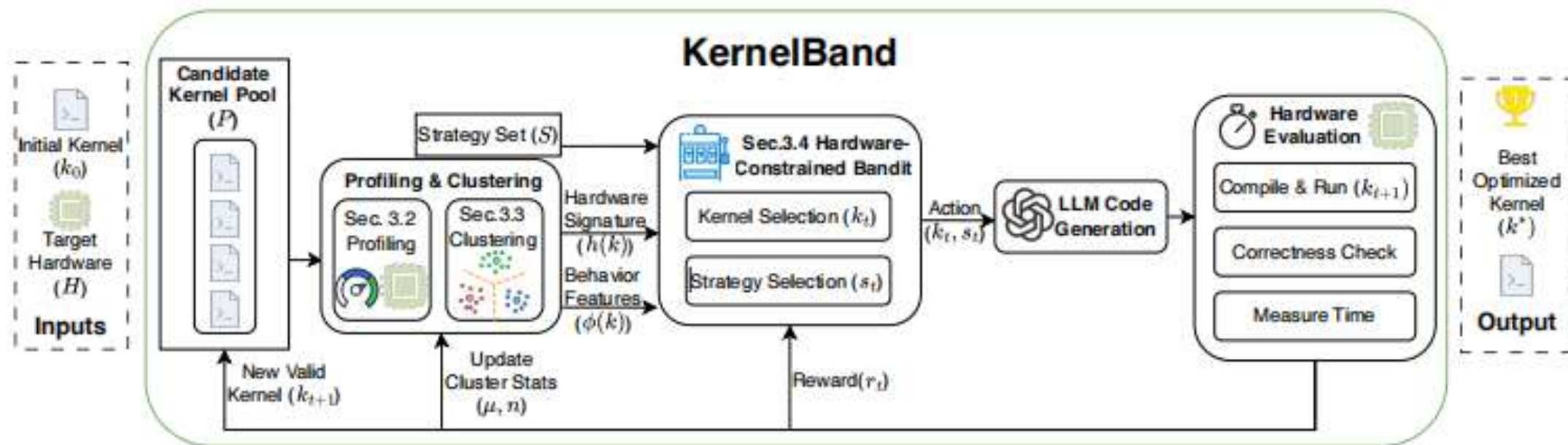
采样方式是基于局部潜力分数的softmax分布, 差值越

Algorithm 1 Workflow of KERNELBAND

```
0: Kernel  $k_0$ , Strategies  $\mathcal{S}$ , Budget  $T$ , Period  $\tau = 10$ , Clusters  $K$ 
1:  $\mathcal{P} \leftarrow \{k_0\}$ ,  $\mathcal{C} \leftarrow \{\mathcal{P}\}$   $\triangleright$  Initial single cluster
2: Initialize  $N_{i,s} \leftarrow 1$ ,  $\hat{\mu}_{i,s} \leftarrow 0.5$  for all  $i, s$ 
3: Initialize Masks  $M_{i,s} \leftarrow 1$   $\triangleright$  No pruning initially
4: for  $t = 1$  to  $T$  do
5:   Compute  $\phi(k)$  for all  $k \in \mathcal{P}$ 
6:    $\triangleright$  Periodic clustering & profiling
7:   if  $t \bmod \tau = 0$  and
8:      $|\mathcal{P}| \geq 2K$  then
9:        $\mathcal{C} \leftarrow \text{KMeans}(\{\phi(k)\}, K)$ 
10:      Update centroids  $k_c^{(t)}$  and profile  $h_c^{(t)} \leftarrow \text{NCU}(k_c^{(t)})$ 
11:    end if
12:     $\triangleright$  Hardware-constrained selection
13:    if centroids are
14:      profiled then
15:         $M_{i,s} \leftarrow \mathbb{I}[h_c^{(t)}[\text{Target}(s)] < \theta_{sat}]$ 
16:      end if
17:      Select  $(I_t, S_t)$  via Eq. (6) (Masked UCB)
18:      Sample  $k_t \in \mathcal{C}_{I_t}$  with  $P(k) \propto \exp(V_{hw}(k, S_t))$ 
19:       $\triangleright$  Generation & update
20:       $k'_t \leftarrow \text{LLM}(k_t, S_t)$ 
21:      if Verify( $k'_t$ ) then
22:         $r_t \leftarrow \max(0, (\mathcal{J}(k_t) - \mathcal{J}(k'_t)) / \mathcal{J}(k_t))$ 
23:         $\mathcal{P} \leftarrow \mathcal{P} \cup \{k'_t\}$ 
24:         $N_{i_t, S_t} \leftarrow N_{i_t, S_t} + 1$ 
```

它把搜索复杂度从庞大的kernel空间, 降低到了聚类数量K和有效策略数量 $|s_{valid}|$ 上。

Design-Overview



profiling进行策略剪枝+聚类 → 上下文老虎机选择kernel与策略 → LLM生成新kernel+更新候选池、聚类状态及奖励

Design-Details

优化策略	描述
Tiling (分块)	将计算划分为可配置大小的tile (计算块), 以提高缓存局部性和并行性。
Vectorization (向量化)	使用向量化加载/存储操作 (例如 float4), 提高内存访问吞吐量。
Fusion (算子融合)	将多个操作合并, 减少中间结果的数据搬运和内存访问开销。
Pipeline (流水线优化)	配置软件流水线深度, 通过隐藏访存延迟提高执行效率。
Reordering (重排序)	优化循环顺序和指令调度, 提高指令级并行性 (ILP) 。
Access & Layout (访问方式与数据布局优化)	优化内存访问模式、合并访问 (coalescing) 以及数据布局。

- 优化策略为六种
- 聚类数量为 $K=3$
- $\tau=10$, 隔10代重聚类
- 硬件饱和阈值 $\theta_{sat}=75\%$
- 探索参数为 $c=2$

Contents

- Background
- Design
- **Evaluation**
- Thinking

Evaluation

实验设置

- 在修正版的TritonBench-G上进行评估。TritonBench-G是一个用于GPU Triton kernel的流行基准测试集合，排除一些简单测试案例后，共发现了183种适用于13种功能类别的kernel，涵盖五个难度级别（L1-L5）。
- 硬件平台方面，我们在三块NVIDIA GPU上进行实验，这些GPU涵盖了消费级（RTX 4090）和数据中心级（H20和A100）硬件。所有平台的运行环境均为CUDA 12.1版本，同时使用了Triton 3.3.0软件。
- LLM后端是DeepSeek-V3.2（DeepSeek AI，2025 年发布）。在后面评估其他后端（如OpenAI、Anthropic、Google 等）的适用性。
- Baselines。(1) GEAK这是一个开源的Triton内核优化工具，采用迭代细化算法，能够轻松适应我们的硬件环境；(2) Best-of-N（BoN）方案，该方案会随机生成N个版本，并选择速度最快的变体。
- 迭代次数T=20，扩展实验中T=40，用于观察收敛速度。
- 评估方法。两阶段验证：(1)调用准确性检查用于检测运行时错误；(2)执行准确性则通过torch.allclose来确保数值上的等效性，通过成功的方案会在10种以上的输入格式下进行性能测试；每任务的速度提升率是指总运行时间对比优化后的总时间。
- 评价指标，(1)Correct (%)：指完成任务中至少有一个有效kernel的所占的比例；(2)Fast@1 (%)：指完成任务中至少有一个速度快于原来版本所占的比例；几何平均性能提升G：有两种报告方式。标准模式仅统计正确的任务以评估优化效果；备用模式则给失败和表现不佳的任务分配一个基准性能提升值1.0×，模拟真实部署，如果优化失败，就退回原 kernel。

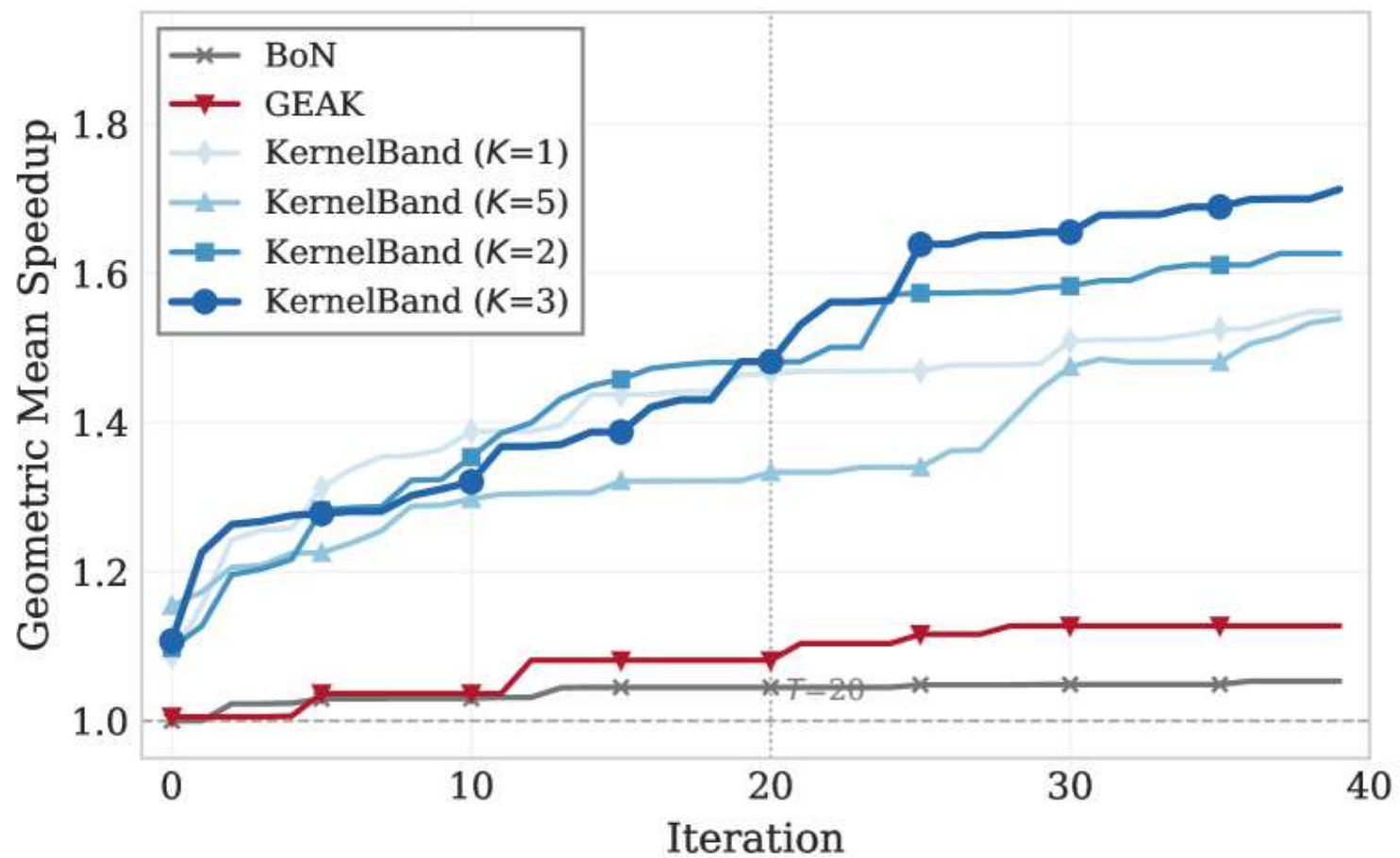
$$G = \left(\prod_{i=1}^N S_i \right)^{\frac{1}{N}}$$

Evaluation

Platform	Method	L1-2			L3			L4-5			All		
		C	F	G	C	F	G	C	F	G	C	F	G
RTX 4090	BoN	63.6	9.1	0.86	12.1	6.1	0.89	28.6	14.3	1.09	31.1	10.0	0.96
	GEAK	68.2	36.4	1.62	21.2	18.2	1.45	80.0	40.0	1.34	55.6	31.1	1.44
	KERNELBAND	81.8	50.0	2.14	60.6	30.3	1.82	91.4	51.4	1.47	77.8	43.3	1.74
H20	BoN	65.5	34.5	1.41	36.9	23.1	1.35	7.1	1.4	0.63	29.3	15.9	0.99
	GEAK	68.9	34.5	1.29	60.0	29.2	1.16	31.4	14.3	0.90	49.4	23.8	1.06
	KERNELBAND	93.1	51.7	1.70	86.2	63.1	1.35	67.1	54.3	1.46	79.3	57.3	1.45
A100	BoN	73.9	34.8	1.43	38.7	24.2	1.23	14.8	6.8	0.76	31.2	16.8	0.98
	GEAK	65.2	43.5	1.09	54.8	45.2	1.18	38.6	33.0	1.54	48.0	38.7	1.34
	KERNELBAND	95.7	82.6	2.23	91.9	77.4	2.05	67.1	42.1	1.75	79.8	60.1	1.91

KernelBand在所有GPU架构上都表现最为出色

Evaluation



聚类数为3时性能最佳

Evaluation

Model	Method	C (%)	F (%)	G
DeepSeek-V3.2	BoN	27.5	12.5	1.10
	GEAK	37.5	17.5	0.95
	KERNELBAND	85.0	67.5	1.52
GPT-5	BoN	44.9	28.6	1.14
	GEAK	51.0	24.5	1.07
	KERNELBAND	81.6	65.3	1.72
Claude Opus 4.5	BoN	40.8	24.5	1.17
	GEAK	63.3	38.8	1.30
	KERNELBAND	89.8	73.5	1.82
Gemini 3 Flash	BoN	47.9	25.0	1.20
	GEAK	62.5	37.5	1.21
	KERNELBAND	70.8	45.8	1.48

Strategy	Freq (%)	Succ (%)	Best (%)
Tiling	10.0	14.4	61.5
Vectorization	14.7	57.1	17.1
Fusion	12.8	75.0	55.2
Pipeline	9.8	64.4	26.3
Reordering	33.2	48.7	25.3
Access & Layout	19.5	29.5	19.2

不同策略的使用频率，以及成功率，最佳贡献率

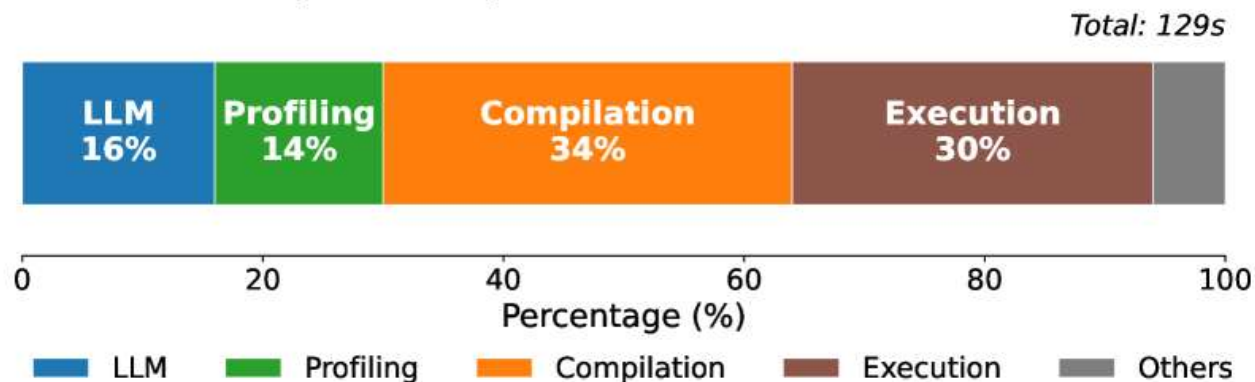
在不同模型上泛化性很好，并且模型越强，优化效果越好

Evaluation

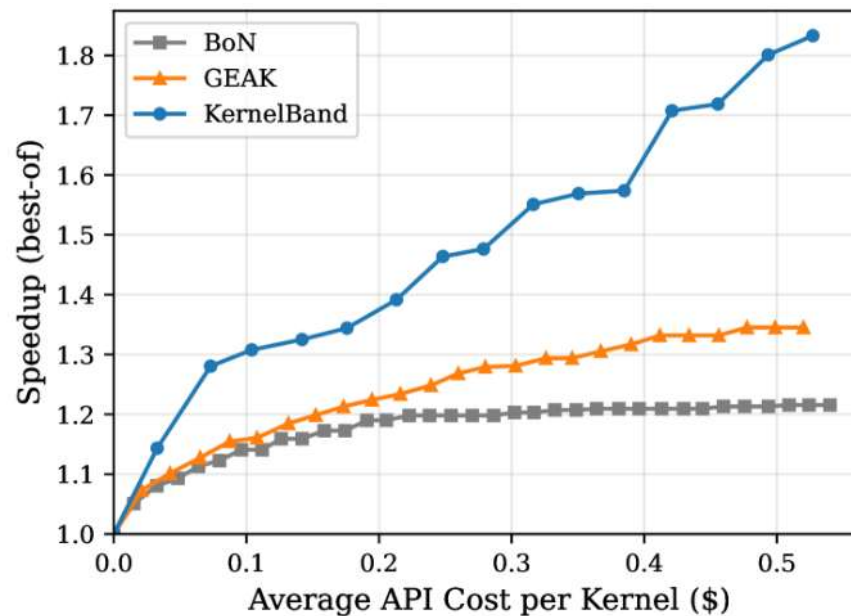
Serial View (Cumulative)



Parallel View (Effective)



串行和并行方式的时间分配



每内核的加速效果与 API 成本之间的关系。在相同的预算下，KernelBand 方案的加速效果高出 35-50%

Evaluation

Type	Configuration	C (%)	F (%)	G	
Single	KernelBand (Full)	87.8	63.4	1.57	
	w/o Clustering ($K=1$)	82.9	58.5	1.41	无聚类, 即 $k=1$
	w/o Profiling	85.4	56.1	1.26	无剪枝,即无掩码
	LLM Strategy Selection	68.3	36.6	0.97	LLM选策略
Frame.	w/o Strategy + Raw Prof.	43.9	26.8	1.12	无策略集, 仅提供profiling信息
	w/o Strategy Set	78.0	48.8	1.15	无策略, 不提供profiling信息
	BoN (baseline)	34.2	17.1	0.60	

消融实验

Contents

- Background
- Design
- Evaluation
- Thinking

Thinking

Conclusion

KernelBand 将基于 LLM 的 kernel 优化从“让模型凭经验反复改代码”，转化为一个由硬件 profiling、动态聚类和多臂老虎机共同指导的结构化搜索过程。

主要贡献：

1. 动态聚类+策略剪枝缩小搜索空间
2. 上下文老虎机平衡探索与利用

局限性：

1. 没有指明重新聚类后平均期望奖励和策略探索次数怎么变化的？
2. 质心profiling代表性可能不够强。
3. 主要还是单kernel优化，没有解决模型级联合优化。

可提升之处：

1. 策略使用只有短短的自然语言描述，能否根据硬件特性细致描述，另外真实kernel往往采用多个策略，所以能否将选策略改为选策略集合。
2. 我的想法是在模型级优化中先使用LLM决定哪个算子或算子组合需要优化，再接kernelband。

泛化性：

1. 聚类的方法能否泛化到其他代码生成任务去减小搜索空间。

Methods QiMeng-Kernel

Macro-Thinking Micro-Coding Paradigm for LLM-Based High-Performance GPU Kernel Generation AAAI'25

自动化生成高性能GPU内核的核心挑战源于 GPU 内核优化空间的极端复杂性+底层硬件特定约束下的实现细节



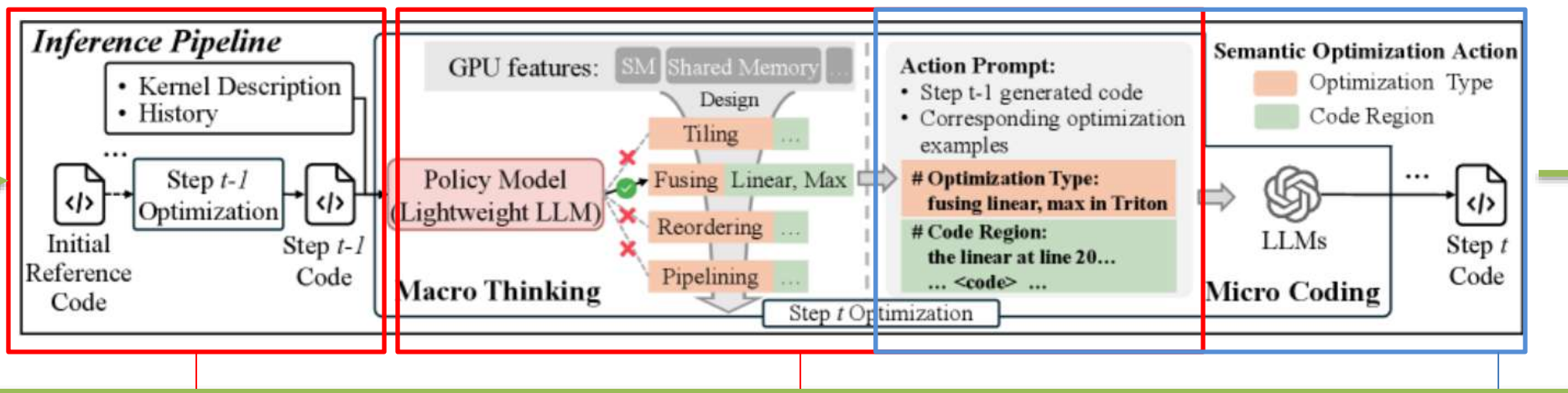
LLM缺乏对硬件和优化行为的精确理解，因此无法导航广阔的优化空间或管理复杂的实现细节



于是将GPU内核生成任务解耦为高级优化策略和逐步优化的代码实现

Methods QiMeng-Kernel

GPU内核生成新范式MTMC，宏观思考，微观编程



待优化的内核描述、
当前步骤内核代码、
历史优化记录、目标
GPU特性

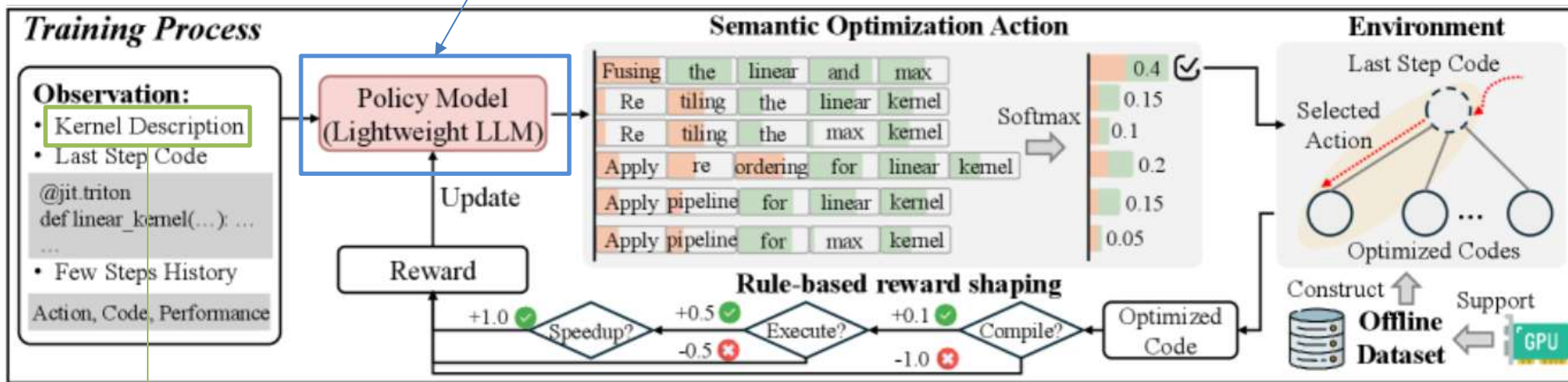
输出语义优化动作，包含优化类型和
代码区域

使用通用模型，输入上一步代码以及相应优化
类型的案例，策略模型的语义优化动作

Methods QiMeng-Kernel

策略模型训练过程

DeepSeek-Coder-1.3B或Llama-3.2-1B



具体任务

语义优化动作空间：1.动作构成（优化类型+候选代码区域）2.系统通过抽象语法树和数据流分析识别候选代码区域

给出选项

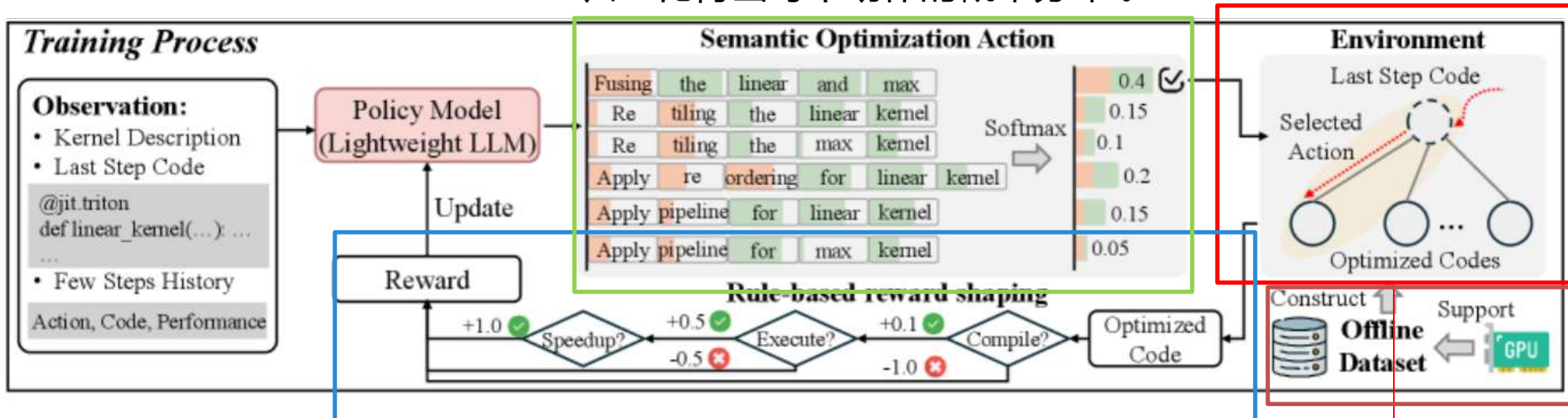
数据集：收集了60k条优化轨迹的离线数据集，涵盖从单个算子、子图到完整神经网络的多样化任务

流程：从数据集抽任务，系统处理初始代码，模型选择语义动作，使用该修改进行测试，根据结果反馈，更新模型。

Methods QiMeng-Kernel

策略模型训练过程

动作被视为一系列 Token 序列，其采样概率等于序列中所有 Token 的联合概率，最后通过 Softmax 归一化得出每个动作的概率分布。



查数据库得性能，无需实际跑，没有则实际执行

阶梯式奖励：编译（语法正确）-执行（逻辑正确）-性能提升，关键引入步长比例奖励衰减，防止无意义的刷分（随优化次数增多，每次优化得分乘一个衰减参数，减分不变）



東南大學
SOUTHEAST UNIVERSITY

Q & A

汇报人：辛金韩

2026年7月13日